

Python - Join Lists

Join Lists in Python

Joining lists in **Python** refers to combining the elements of multiple lists into a single **list**. This can be achieved using various methods, such as concatenation, list comprehension, or using built-in functions like `extend()` or `+` operator.

Joining lists does not modify the original lists but creates a new list containing the combined elements.

Join Lists Using Concatenation Operator

The concatenation operator in Python, denoted by `+`, is used to join two sequences, such as strings, lists, or tuples, into a single sequence. When applied to lists, the concatenation operator joins the elements of the two (or more) lists to create a new list containing all the elements from both lists.

We can join a list using the concatenation operator by simply using the `+` symbol to concatenate the lists.

Example

In the following example, we are concatenating the elements of two lists "L1" and "L2", creating a new list "joined_list" containing all the elements from both lists –

```
# Two lists to be joined
L1 = [10, 20, 30, 40]
L2 = ['one', 'two', 'three', 'four']
# Joining the lists
joined_list = L1 + L2
```

```
# Printing the joined list
print("Joined List:", joined_list)
```

Following is the output of the above code –

```
Joined List: [10, 20, 30, 40, 'one', 'two', 'three', 'four']
```

Join Lists Using List Comprehension

List comprehension is a concise way to create lists in Python. It is used to generate new lists by applying an expression to each item in an existing iterable, such as a list, tuple, or range. The syntax for list comprehension is –

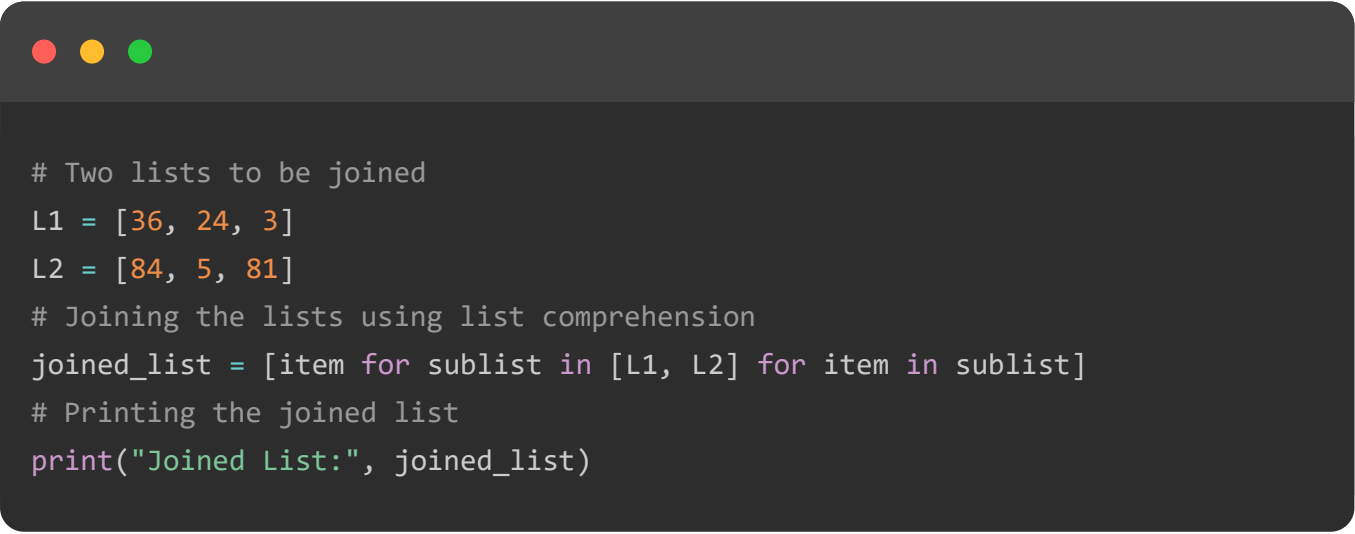
```
new_list = [expression for item in iterable]
```

This creates a new list where **expression** is evaluated for each **item** in the **iterable**.

We can join a list using list comprehension by iterating over multiple lists and appending their elements to a new list.

Example

In this example, we are joining two lists, L1 and L2, into a single list using list comprehension. The resulting list, `joined_list`, contains all elements from both L1 and L2 –



```
# Two lists to be joined
L1 = [36, 24, 3]
L2 = [84, 5, 81]
# Joining the lists using list comprehension
joined_list = [item for sublist in [L1, L2] for item in sublist]
# Printing the joined list
print("Joined List:", joined_list)
```

Output of the above code is as follows –

```
Joined List: [36, 24, 3, 84, 5, 81]
```

Join Lists Using append() Function

The `append()` function in Python is used to add a single element to the end of a list. This function modifies the original list by adding the element to the end of the list.

We can join a list using the `append()` function by iterating over the elements of one list and appending each element to another list.

Example

In the example below, we are appending elements from "list2" to "list1" using the `append()` function. We achieve this by iterating over "list2" and adding each element to "list1" –

```
# List to which elements will be appended
list1 = ['Fruit', 'Number', 'Animal']
# List from which elements will be appended
list2 = ['Apple', 5, 'Dog']
# Joining the lists using the append() function
for element in list2:
    list1.append(element)
# Printing the joined list
print("Joined List:", list1)
```

We get the output as shown below –

```
Joined List: ['Fruit', 'Number', 'Animal', 'Apple', 5, 'Dog']
```

Join Lists Using extend() Function

The Python `extend()` function is used to append elements from an iterable (such as another list) to the end of the list. This function modifies the original list in place, adding the elements of the iterable to the end of the list.

We can join a list using the `extend()` function by calling it on one list and passing another list (or any iterable) as an argument. This will append all the elements from the second list to the end of the first list.

Example

In the following example, we are extending "list1" by appending the elements of "list2" using the extend() function –

```
# List to be extended
list1 = [10, 15, 20]
# List to be added
list2 = [25, 30, 35]
# Joining the lists using the extend() function
list1.extend(list2)
# Printing the extended list
print("Extended List:", list1)
```

The output obtained is as shown below –

```
Extended List: [10, 15, 20, 25, 30, 35]
```

TOP TUTORIALS

- Python Tutorial
- Java Tutorial
- C++ Tutorial
- C Programming Tutorial
- C# Tutorial
- PHP Tutorial
- R Tutorial
- HTML Tutorial
- CSS Tutorial
- JavaScript Tutorial
- SQL Tutorial

TRENDING TECHNOLOGIES

- Cloud Computing Tutorial

Amazon Web Services Tutorial

Microsoft Azure Tutorial

Git Tutorial

Ethical Hacking Tutorial

Docker Tutorial

Kubernetes Tutorial

DSA Tutorial

Spring Boot Tutorial

SDLC Tutorial

Unix Tutorial

CERTIFICATIONS

Business Analytics Certification



Chapters ▾

Categories ≡

Cloud Computing And DevOps

Advanced Certification In Business Analytics

Artificial Intelligence And Machine Learning

DevOps Certification

Game Development Certification

Front-End Developer Certification

AWS Certification Training

Python Programming Certification

COMPILERS & EDITORS

Online Java Compiler

Online Python Compiler

Online Go Compiler

Online C Compiler

Online C++ Compiler

Online C# Compiler

Online PHP Compiler

Online MATLAB Compiler

Online Bash Terminal

Online SQL Compiler

Online Html Editor

[ABOUT US](#) | [OUR TEAM](#) | [CAREERS](#) | [JOBS](#) | [CONTACT US](#) | [TERMS OF USE](#) |
[PRIVACY POLICY](#) | [REFUND POLICY](#) | [COOKIES POLICY](#) | [FAQ'S](#)



Tutorials Point is a leading Ed Tech company striving to provide the best learning material on technical and non-technical subjects.

© Copyright 2025. All Rights Reserved.